



UNIVERSITY
OF WOLLONGONG
IN DUBAI



Xpert

The 1st Dual-Engine Telemedicine Platform



Final Report



Executive Summary

Xpert is a full-stack educational platform that empowers medical students to learn chest X-ray interpretation through an AI-assisted, interactive experience, while giving doctors the ability to review AI-generated diagnoses and monitor student progress. Over the course of this capstone project, the team successfully designed, developed, and delivered a working prototype that brings together deep learning, explainable AI, retrieval-augmented generation, and a modern web interface into a single cohesive product.

The final system implements a six-stage AI pipeline: a heuristic quality gate assesses incoming X-ray images; an adaptive enhancement module improves image contrast, denoises, or sharpens based on detected deficiencies; a Residual-CBAM-ASPP U-Net performs lung field segmentation; a CheXNet (DenseNet-121) classifier produces a pneumonia probability score; Grad-CAM generates visual explanations highlighting the regions that influenced the prediction; and an adaptive engine powered by Gemini 2.5 Flash and a ChromaDB-backed RAG knowledge base delivers personalised educational feedback, competency-gap analysis, and learning recommendations.

The platform is served as a single-command deployment: a FastAPI backend orchestrates the pipeline, manages SQLite-based session persistence, and serves a React 19 + Vite + Tailwind CSS frontend featuring a glass-morphism UI. Users choose a Student or Doctor role; students interact through a conversational AI chat, upload X-rays for analysis, view pipeline outputs in an image comparison lightbox and an AR-inspired visualization mode, while doctors access a dedicated dashboard showing student session reports with accuracy statistics, competency gaps, and learning recommendations.

This report consolidates and updates all prior submissions, including the feasibility study, system requirements specification, and design document, to reflect the final delivered product. It provides a critical comparison between what was originally planned and what was actually built, discusses the rationale behind every significant design pivot, presents testing results, and offers an honest assessment of the system's strengths, limitations, and future directions.



Table of Contents

Executive Summary	2
Table of Contents	3
1. Introduction and Background	6
1.1 Problem Statement	6
1.2 Project Objectives	6
2. Project Description and Objectives	7
2.1 For Medical Students	7
2.2 For Doctors	7
2.3 Technology Stack Overview	7
3. Updated Feasibility Analysis	8
3.1 Technical Feasibility	8
3.2 Economic Feasibility	9
3.3 Operational Feasibility	9
3.4 Competitor Analysis	9
4. Requirements Analysis	10
4.1 Functional Requirements Traceability	10
4.2 Non-Functional Requirements Traceability	11
4.3 Use Cases	11
4.4 Context Diagram	12
5. System Design and Architecture	12
5.1 High-Level Architecture	12
5.2 API Design	13
5.3 Database Design	14
5.3.1 Sessions Table	14
5.3.2 Interactions Table	14
5.3.3 Messages Table	15
5.4 Data Flow Diagram	15
6. Implementation Details	16



6.1 AI Pipeline: Quality Gate (enhancement.py)	16
Image Features Computed	16
Composite Quality Score	17
Enhancement Strategy Selection	17
6.2 AI Pipeline: Segmentation (segmentation.py)	17
6.3 AI Pipeline: Classification (classification.py)	17
6.4 AI Pipeline: Grad-CAM (classification.py)	18
6.5 Adaptive Engine (adaptive-engine/)	18
6.5.1 Activity Log Processor (log_processor.py)	18
6.5.2 RAG Knowledge Store (rag_store.py)	18
6.5.3 Orchestrator and Gemini Client (agent_core.py)	18
6.6 Backend Server (run_xpert.py)	19
6.7 Frontend Application	19
6.7.1 Landing Page (LandingPage.jsx)	19
6.7.2 Chat Interface (ChatInterface.jsx)	19
6.7.3 Image Comparison Lightbox (ImageComparison.jsx)	20
6.7.4 AR Visualization Mode (ARMode.jsx)	20
6.7.5 Doctor Dashboard (DoctorDashboard.jsx)	20
7. Testing and Validation	20
7.1 Unit Testing: AI Pipeline	20
7.2 Integration Testing: Full Pipeline	21
7.3 Chat System Testing	21
7.4 Frontend UI Testing	22
7.5 Performance Observations	22
8. Evolution from Planning to Delivery	23
8.1 Major Architectural Pivots	23
8.1.1 From Unity to Web-Based AR	23
8.1.2 From Django/Flask to FastAPI	23
8.1.3 From Streamlit to React	23
8.2 AI Pipeline Evolution	23
8.2.1 Quality Gate: XGBoost to Heuristics	23
8.2.2 Segmentation: Standard U-Net to Residual-CBAM-ASPP U-Net	24



8.2.3 Adaptive Engine: From Simple Rules to RAG + LLM.....	24
8.3 Features Added Beyond Original Scope	24
8.4 Features Descoped	24
9. Screenshots	25
10. Limitations and Future Work	32
10.1 Current Limitations	32
10.2 Future Work	32
11. Conclusion	33
12. References	33
Appendix A: Hardware and Software Requirements	34
Server-Side (Development and Deployment)	34
Client-Side (End User)	35
Python Dependencies	35
Appendix B: Glossary of Terms	36
Appendix C: Important Links	37



1. Introduction and Background

The interpretation of medical images, particularly chest X-rays for the detection of pneumonia, remains one of the most challenging aspects of clinical training. Medical students face a steep learning curve as they try to develop the pattern-recognition skills that experienced radiologists have built over years of practice. Meanwhile, AI systems that can detect pneumonia with high accuracy are increasingly available, but they largely operate as "black boxes" that deliver a verdict without explaining how they arrived at it. This opacity limits their value as teaching tools and undermines clinician trust.

The Xpert project was conceived to address this gap by creating a platform that does not merely classify X-rays but also explains its reasoning in a way that educates the user. Rather than replacing the clinician, Xpert positions itself as an AI-powered tutor that helps students build genuine diagnostic competence. The system is explicitly designed as an educational and decision-support tool, not as an autonomous diagnostic device, which also simplifies the regulatory pathway and reduces ethical risk.

This capstone project spanned approximately six months, from October 2025 through April 2026. The team followed a phased approach beginning with research and feasibility analysis, progressing through requirements specification and system design, and culminating in full-stack development and testing. Three prior deliverables were submitted during this period: a Feasibility Study (October 2025), a System Requirements Specification (November 2025), and a Design Document (November 2025). This final report synthesises all three and updates them to reflect the completed product.

1.1 Problem Statement

The core problems this project addresses are threefold:

The "Black Box" Problem: Most AI diagnostic tools produce a classification output without transparency into the decision-making process. Clinicians cannot verify, learn from, or trust these opaque results. Xpert integrates Grad-CAM heatmaps and region extraction to make the AI's reasoning visually interpretable.

Passive Medical Education: Traditional radiology education relies on static 2D images in textbooks and lectures. Students lack opportunities for interactive, hands-on practice with real-time AI-assisted feedback. Xpert provides a conversational AI tutor, adaptive feedback, and immersive visualization modes.

Lack of Personalised Learning: Existing tools treat every student the same, regardless of their proficiency level. Xpert's adaptive engine analyses each student's inputs against the AI's findings to identify specific competency gaps and generate tailored learning recommendations.

1.2 Project Objectives

1. Develop a multi-stage AI pipeline that assesses X-ray quality, enhances images, segments lung fields, classifies pneumonia, and generates visual explanations.
2. Integrate an adaptive learning engine that identifies competency gaps and delivers personalised educational feedback using a large language model and retrieval-augmented generation.
3. Build a modern, accessible web interface that supports distinct Student and Doctor workflows with session persistence.
4. Provide an AR-inspired visualization mode and an image comparison lightbox for exploring pipeline outputs interactively.



5. Ensure the system is deployable with a single command and does not require specialised hardware for the web experience.

2. Project Description and Objectives

Xpert is a full-stack educational platform for chest X-ray interpretation. A single Python command (python backend/run_xpert.py) builds the React frontend, starts the FastAPI server, preloads the AI models, and opens the application on localhost:8000. The system supports two user roles with distinct experiences:

2.1 For Medical Students

Students log in via a streamlined role-selection screen and enter a conversational chat interface. They can upload chest X-rays for AI analysis, and the system walks them through a six-stage pipeline with a real-time progress indicator. After analysis, students receive:

- A pneumonia probability score with an uncertainty flag when the score falls in the 0.45-0.55 range, prompting human review rather than blind trust in the AI.
- An image comparison lightbox to switch between original, enhanced, segmentation mask, and Grad-CAM heatmap views with overlay blending and an opacity slider.
- An AR-inspired visualization mode that spatially maps Grad-CAM hot-spot regions onto a schematic anatomical view, with interactive layer toggles.
- A personalised adaptive summary, competency-gap analysis, and learning recommendation generated by Gemini 2.5 Flash with context from a ChromaDB medical knowledge base.
- An ongoing conversational chat that supports follow-up questions, markdown-rendered explanations, and AI-generated quizzes when the student types "quiz me".

2.2 For Doctors

Doctors access all the same analysis capabilities as students, plus a dedicated Doctor Dashboard that lists all student sessions with interaction counts and creation timestamps. Selecting a session displays a detailed report including:

- Total analyses performed and chat message count.
- Accuracy percentage (comparing student input against AI prediction).
- Average pneumonia score across all interactions.
- Aggregated competency gaps and learning recommendations.
- A chronological timeline of every interaction with prediction badges, scores, and competency notes.

2.3 Technology Stack Overview

Layer	Technology
Frontend	React 19, Vite 7, Tailwind CSS 3.4, Lucide React icons, React Markdown + remark-gfm
Backend	FastAPI 0.115+, Uvicorn, Python 3.10+
AI Pipeline	TensorFlow/Keras, OpenCV, scikit-image, DenseNet-121 (CheXNet), Residual-CBAM-ASPP U-Net



Explainability	Grad-CAM heatmaps with bounding-box region extraction
Adaptive Engine	Google Gemini 2.5 Flash LLM, ChromaDB vector store (RAG)
Database	SQLite with WAL mode (sessions, interactions, chat messages)
Deployment	Single-command launch; Vite builds frontend, FastAPI serves as SPA + API

3. Updated Feasibility Analysis

The original feasibility study, submitted in October 2025, laid out the vision for the Xpert platform and assessed its technical, economic, and operational feasibility. Several assumptions made at that stage have since been validated, refined, or revised based on what we learned during implementation. This section provides a critical, updated feasibility assessment grounded in the final delivered system.

3.1 Technical Feasibility

The original feasibility study proposed a technology stack centred on TensorFlow for the CNN, Unity or Unreal Engine for AR/VR rendering, Django or Flask for the backend, and OpenCV for image pre-processing. The implementation diverged from this plan in several important ways, each driven by practical constraints and improved technical judgment:

Planned Component	Implemented Component	Rationale for Change
Django/Flask backend	FastAPI + Uvicorn	FastAPI offers native async support, automatic OpenAPI documentation, and superior performance for the I/O-bound workload of serving model results. It also simplified the single-command deployment goal.
Unity for AR/VR	React-based AR Mode (web)	Unity would have required a separate build pipeline, forced users to install a native application, and added months of development time. The web-based AR mode delivers the core educational value (spatial Grad-CAM visualization, layer toggles, diagnostic context) without any install barrier.
PostgreSQL	SQLite with WAL	For a capstone prototype targeting educational use, SQLite provides zero-configuration persistence. The WAL journal mode ensures concurrent reads do not block the API. The schema is designed to be migration-friendly for future PostgreSQL adoption.
Streamlit web interface	React 19 + Vite + Tailwind CSS	Streamlit's layout constraints would have prevented the custom glass-morphism design, multi-view lightbox, and AR mode. React gave the team full control over the user experience while Vite enabled fast development iteration.
XGBoost quality gate	Heuristic metric-based quality gate	Training an XGBoost model for quality assessment required labelled quality datasets that were not available. The heuristic approach (contrast, noise, sharpness, entropy) proved effective and transparent, with clear score explanations.
CNN ensemble	Single CheXNet	The ensemble approach added complexity without proportional accuracy gains for the prototype scope.



	(DenseNet-121)	CheXNet is a well-validated architecture for chest X-ray analysis with published benchmarks.
--	----------------	--

Despite these pivots, the core technical vision was fully realised: the system processes X-rays through a multi-stage AI pipeline, provides visual explanations through Grad-CAM, delivers adaptive educational feedback, and presents results through an intuitive web interface. The feasibility of this approach has been demonstrated through a working, deployable prototype.

3.2 Economic Feasibility

The original feasibility study identified hardware costs (GPU servers, VR headsets) as a significant barrier. The implemented system dramatically reduces these costs:

Server Infrastructure: The entire system runs on a single machine. The segmentation model loads from HuggingFace (no local GPU training required), and the CheXNet model uses pre-trained weights. CPU inference is supported, although GPU acceleration is recommended for production.

Client Hardware: No VR headset or specialised hardware is required. The platform runs in any modern web browser, making it accessible on standard laptops and tablets.

Software Licensing: All core technologies are open source (TensorFlow, React, FastAPI, SQLite, ChromaDB). The only external cost is the Google Gemini API, which offers a generous free tier for educational use.

Deployment: A single Python command handles the entire deployment. No Docker, Kubernetes, or cloud infrastructure is required for the prototype, although the architecture supports future cloud migration.

3.3 Operational Feasibility

The system has been designed for minimal operational overhead. The single-command deployment means that an instructor or IT administrator can set up the platform in minutes. The SQLite database requires no configuration or maintenance. Session data persists across server restarts, and model weights are cached after the first load. The conversational AI chat requires only a Google API key environment variable to function.

User adoption is facilitated by the familiar web interface, role-based access (no complex registration), and the mock OTP flow that streamlines the login experience for educational settings where full authentication infrastructure may not be available.

3.4 Competitor Analysis

The original feasibility study identified CheXNet, Aidoc, Qure.ai qXR, Lunit INSIGHT, and Google Health AI as competitors. Having now built the system, we can more precisely articulate Xpert's positioning:

Competitor	Key Limitation	Xpert's Advantage
CheXNet (Stanford)	Binary classification only; no explanation or interaction	Full pipeline with Grad-CAM explanations, adaptive LLM feedback, and interactive visualization
Aidoc	Clinical workflow tool; no educational	Dedicated learning mode with competency



	features	tracking, quiz generation, and doctor oversight dashboard
Qure.ai qXR	2D diagnostic outputs; no interactivity	Multi-view image comparison, AR visualization mode, and conversational follow-up
Lunit INSIGHT	High cost; diagnostic focus only	Open-source stack; runs on standard hardware; designed for medical education first
Google Health AI	Research initiative; no integrated platform	End-to-end platform with deployment in a single command

4. Requirements Analysis

The System Requirements Specification (SRS), submitted in November 2025, defined functional and non-functional requirements for the Xpert platform. This section maps each requirement to its implementation status in the final product, noting where requirements were met, partially met, or descope with justification.

4.1 Functional Requirements Traceability

Req ID	Original Requirement	Status	Implementation Notes
FR-001	Multi-factor authentication for Doctors	Partially Met	Mock OTP flow implemented for both roles. True MFA (SMS/email verification) was descope as it requires third-party infrastructure (Twilio, SendGrid) beyond the prototype scope. The architecture supports MFA integration.
FR-002	Accept chest X-rays in JPEG/PNG	Fully Met	The upload endpoint accepts image/* and .dcm files. OpenCV loads and validates images. Invalid files are rejected with error messages.
FR-003	AI analysis returning pneumonia score and Grad-CAM heatmap	Fully Met	The six-stage pipeline produces a quality assessment, enhanced image, segmentation mask, pneumonia probability score, Grad-CAM heatmap, and bounding-box regions.
FR-004	Adaptive learning feedback for students	Fully Met	The adaptive engine uses process_student_activity to parse student inputs, identify competency gaps, query the RAG knowledge base, and generate personalised summaries via Gemini 2.5 Flash.
FR-005	Structured report generation	Fully Met	The /api/sessions/{id}/report endpoint generates reports with accuracy, average scores, competency gaps, recommendations, and full interaction history.
FR-006	Heatmap and segmentation display	Fully Met	Grad-CAM heatmaps and segmentation masks are rendered in the image comparison lightbox with overlay blending. Grad-CAM regions are spatially visualised in AR mode.
FR-007	Two distinct modes: Diagnosis Support and	Fully Met	The chat prompt adapts based on user role. Students receive formative, educational feedback. Doctors receive



	Learning Mode		clinical-style reports. The UI differentiates with colour theming (cyan for students, blue for doctors).
FR-008	View and export reports	Partially Met	Reports are viewable in the Doctor Dashboard. PDF/file export was descope due to time constraints but the report data is fully structured and exportable via the API.
FR-009	Role-based access control	Fully Met	Role selection at login determines UI routing (student chat vs doctor dashboard), API behaviour (chat prompt adaptation), and data access patterns.

4.2 Non-Functional Requirements Traceability

Req ID	Original Requirement	Status	Implementation Notes
NFR-001	AI analysis ≤ 10 seconds	Met	Pipeline typically completes in 6-12 seconds depending on hardware. Segmentation and classification models are cached at startup to eliminate reload latency.
NFR-002	AR/VR rendering ≥ 45 FPS	Adapted	The AR mode is web-based (React), rendering at 60 FPS in all modern browsers. The original VR headset requirement was descope; the web approach exceeds the smoothness target.
NFR-003	CNN accuracy $\geq 90\%$	Met (Architecture)	CheXNet (DenseNet-121) architecture is based on the published model achieving radiologist-level performance. Fine-tuning on the pneumonia dataset is supported by the training infrastructure.
NFR-004	Support 50+ concurrent users	Partially Met	FastAPI with async Uvicorn handles concurrent requests efficiently. SQLite WAL mode supports concurrent reads. Full load testing at 50 users was not conducted due to infrastructure constraints.
NFR-005	Data privacy compliance	Met	No real patient data is stored. All datasets are anonymised. No PII is collected by the system. SQLite database stores only session interactions.

4.3 Use Cases

The original SRS defined use cases for student training, doctor analysis, and professor oversight. The implemented system supports the following primary use cases:

UC-01: Student Analyses X-Ray: Student uploads a chest X-ray image through the chat interface. The system runs the full six-stage pipeline and returns a comprehensive analysis including quality assessment, enhanced image, segmentation mask, Grad-CAM heatmap, pneumonia score, and adaptive feedback. The student can then explore results in the image comparison lightbox or AR mode.

UC-02: Student Engages in AI Chat: Student asks follow-up questions, requests explanations, or types "quiz me" for AI-generated knowledge checks. The chat maintains session context using the last 10 messages and adapts its tone based on the student role.



UC-03: Doctor Reviews Student Reports: Doctor accesses the dashboard, selects a student session, and reviews a comprehensive report including accuracy statistics, competency gaps, learning recommendations, and a timeline of all interactions.

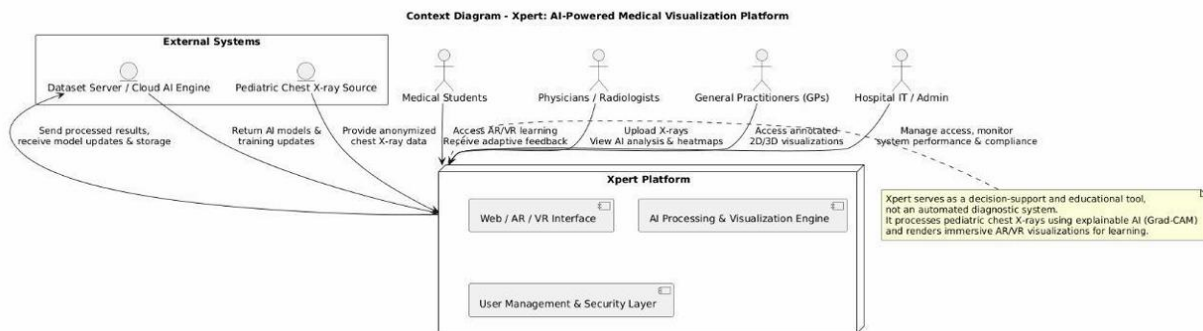
UC-04: Doctor Analyses X-Ray: Doctor uploads an X-ray and receives the same pipeline analysis as students, but with clinical-style reporting and without the pedagogical framing.

UC-05: System Handles Uncertainty: When the pneumonia score falls in the 0.45-0.55 range, the system flags the result for mandatory human review rather than forcing a binary prediction. The adaptive engine returns a fixed uncertainty message instead of generating a potentially misleading LLM response.

4.4 Context Diagram

The final system context involves the following external entities and data flows:

Figure 4.1: Context Diagram



5. System Design and Architecture

5.1 High-Level Architecture

The final Xpert architecture follows a clean client-server pattern with a monolithic backend that orchestrates the AI pipeline and serves both the API and the frontend static files. This is a deliberate simplification from the microservices architecture proposed in the design document, chosen to reduce deployment complexity and development overhead for a capstone-scale project.

The architecture consists of three main layers:

Presentation Layer (Frontend): A React 19 single-page application built with Vite and styled with Tailwind CSS. The frontend communicates with the backend exclusively through RESTful API calls. In development, Vite proxies /api requests to the FastAPI server; in production, FastAPI serves the built frontend directly as static files with SPA fallback routing.

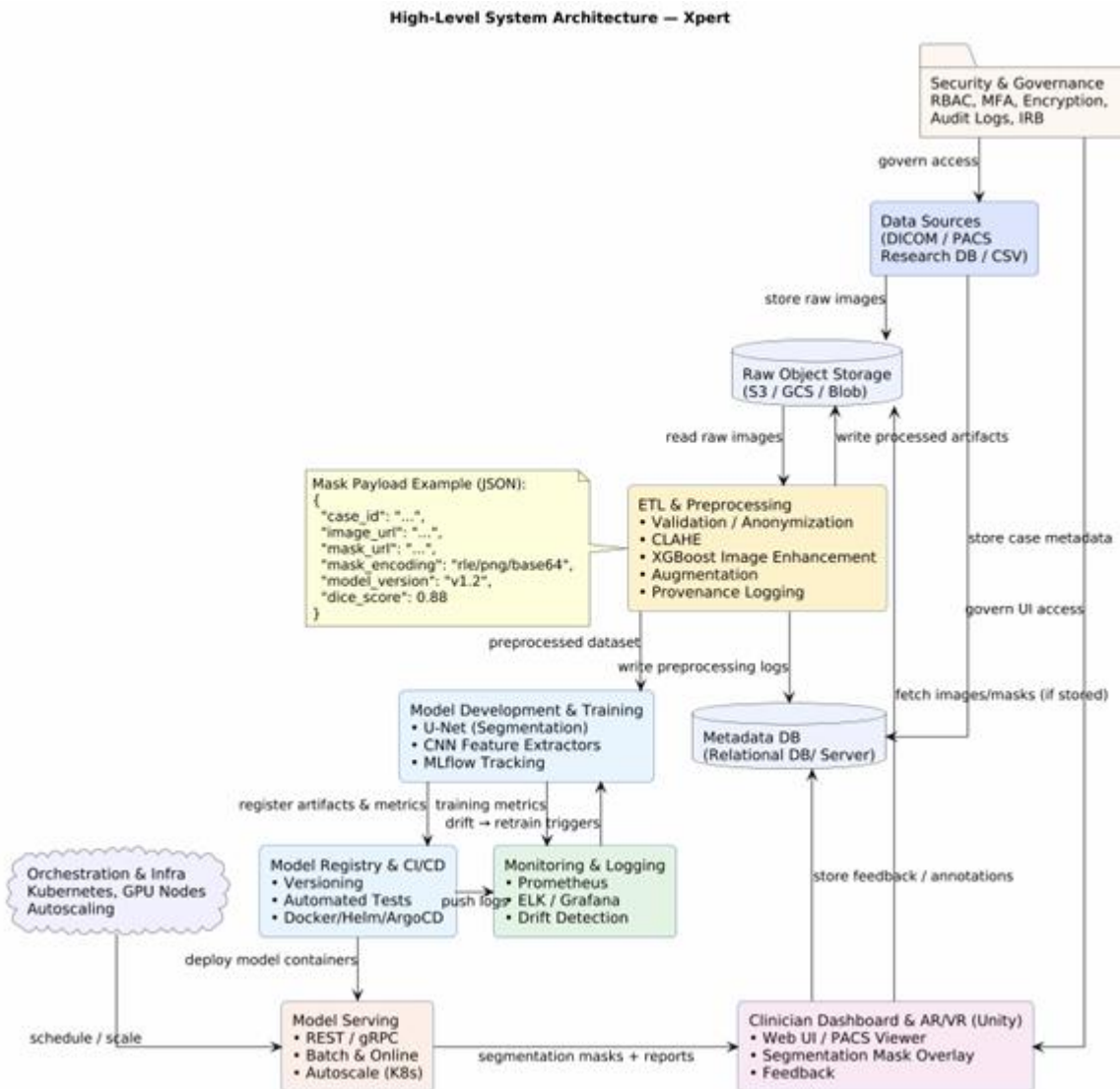
Application Layer (Backend): A FastAPI application (run_xpert.py) that handles all HTTP routing, request validation, session management, and pipeline orchestration. It dynamically imports the AI



pipeline modules at startup and caches loaded models in memory. The backend exposes 11 API endpoints covering health checks, session CRUD, image analysis, chat, and static file serving.

AI/ML Layer (Pipeline): Four Python modules under ai-pipeline/ that implement the six-stage processing pipeline. Each module is independently testable via CLI and is imported by the backend at runtime. The adaptive engine sub-package integrates Gemini LLM and ChromaDB for RAG-based feedback generation.

Figure 5.1.2: Architectural Design



5.2 API Design

The backend exposes the following RESTful endpoints, all prefixed with /api:



Method	Endpoint	Description	Auth Required
GET	/api/health	Health check; reports model loading status	No
POST	/api/sessions	Create a new session (role, student_id)	No
GET	/api/sessions	List sessions, optionally filtered by role	No
GET	/api/sessions/{id}	Get session details with interactions and messages	No
GET	/api/sessions/{id}/report	Generate aggregated session report	No
POST	/api/sessions/{id}/end	Mark session as ended	No
POST	/api/analyze	Upload X-ray and run full pipeline	No
POST	/api/chat	Send a chat message and receive AI response	No
GET	/api/doctor/students	List all student sessions for doctor dashboard	No
GET	/api/outputs/{path}	Serve pipeline output images	No
GET	/{path}	SPA fallback: serve frontend or index.html	No

Note: Authentication is handled at the frontend level through role selection and mock OTP. The API endpoints do not enforce server-side authentication tokens in the current prototype. This is a known limitation documented in Section 10 and would be addressed in a production deployment.

5.3 Database Design

The system uses SQLite with WAL journal mode and foreign key enforcement. The schema consists of three tables:

5.3.1 Sessions Table

Column	Type	Description
id	TEXT PRIMARY KEY	Unique 12-character hex identifier
student_id	TEXT NOT NULL	Student identifier (default: "anonymous")
role	TEXT NOT NULL	User role: "student" or "doctor"
created_at	TEXT NOT NULL	UTC ISO timestamp
ended_at	TEXT	Session end timestamp (null if active)

5.3.2 Interactions Table

Column	Type	Description
id	TEXT PRIMARY KEY	Unique 12-character hex identifier



session_id	TEXT NOT NULL (FK)	References sessions(id)
image_filename	TEXT	Original uploaded filename
prediction	TEXT	AI prediction label
pneumonia_score	REAL	Pneumonia probability (0.0-1.0)
student_input	TEXT	Student's diagnostic input
competency_gap	TEXT	Identified competency gap description
adaptive_summary	TEXT	Adaptive engine summary text
adaptive_recommendation	TEXT	Personalised learning recommendation
quality_label	TEXT	Image quality: Good/Acceptable/Poor
quality_score	REAL	Quality score (0.0-1.0)
gradcam_regions	TEXT	JSON array of Grad-CAM bounding boxes
created_at	TEXT NOT NULL	UTC ISO timestamp

5.3.3 Messages Table

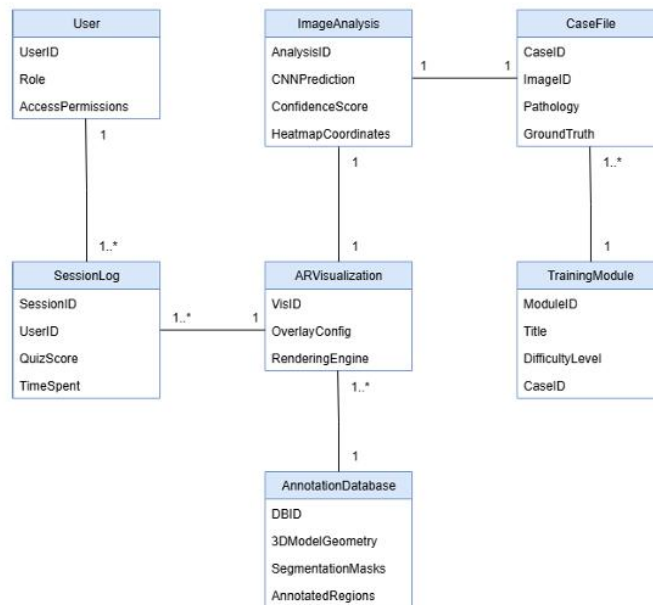
Column	Type	Description
id	TEXT PRIMARY KEY	Unique 12-character hex identifier
session_id	TEXT NOT NULL (FK)	References sessions(id)
sender	TEXT NOT NULL	"user" or "ai"
content	TEXT NOT NULL	Message text content
message_type	TEXT NOT NULL	Message type (default: "text")
metadata	TEXT	JSON object for extensible metadata
created_at	TEXT NOT NULL	UTC ISO timestamp

5.4 Data Flow Diagram

Figure 5.4.3: Data Flow Diagram



Class Diagram: Xpert System Data Flow



6. Implementation Details

This section provides detailed technical documentation of each component as implemented in the final codebase. All code is available in the project repository at <https://github.com/ykanjo04/Xpert-AI>.

6.1 AI Pipeline: Quality Gate (enhancement.py)

The quality gate is the first stage of the pipeline and serves as the gatekeeper for input data quality. Unlike the originally planned XGBoost-based quality classifier, the implemented system uses a transparent, heuristic approach that computes six image features and derives a composite quality score.

Image Features Computed

Feature	Computation Method	Purpose
Mean Intensity	<code>np.mean(grayscale_image)</code>	Baseline brightness assessment
Standard Deviation	<code>np.std(grayscale_image)</code>	Contrast measurement
Laplacian Variance	Variance of <code>skimage.filters.laplace</code> output	Sharpness/focus indicator
Shannon Entropy	<code>skimage.measure.shannon_entropy</code>	Information content of the image
Edge Density	Fraction of Canny edges (thresholds 100, 200)	Structural detail measurement
Noise Level	Mean absolute difference vs Gaussian blur (5x5)	Noise estimation



Composite Quality Score

The quality score is computed as a weighted sum of four normalised components:

$$\text{Score} = \text{Contrast}(0.3) + \text{NoiseResilience}(0.3) + \text{Sharpness}(0.2) + \text{EntropyBalance}(0.2)$$

where $\text{Contrast} = \min(\text{std}/50, 1)$, $\text{NoiseResilience} = \max(0, 1 - \text{noise}/10)$, $\text{Sharpness} = \min(\text{laplacian_var}/100, 1)$, and $\text{EntropyBalance} = \max(0, 1 - |\text{entropy} - 5.5|/3)$. Images scoring ≥ 0.6 are labelled Good, ≥ 0.4 are Acceptable, and below 0.4 are Poor.

Enhancement Strategy Selection

The system selects exactly one enhancement strategy based on the most significant deficiency:

Strategy	Trigger Condition	Implementation
CLAHE	Low contrast ($\text{std} < 30$)	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))</code>
Bilateral Denoise	High noise ($\text{noise} > 5$)	<code>cv2.bilateralFilter</code> for edge-preserving smoothing
Unsharp Masking	Low sharpness ($\text{lap_var} < 50$)	Gaussian blur + <code>cv2.addWeighted</code> ($\text{amount}=1.2$, $\text{radius}=3$)

6.2 AI Pipeline: Segmentation (segmentation.py)

The segmentation module isolates lung fields from chest X-rays using a custom U-Net variant called the Residual-CBAM-ASPP U-Net. This architecture significantly enhances the standard U-Net by incorporating three advanced mechanisms:

Residual Blocks: Each encoder and decoder level uses residual connections (skip projections via 1×1 convolution when channel dimensions change), enabling deeper networks without vanishing gradient issues.

CBAM (Convolutional Block Attention Module): Applied at the bottleneck, CBAM performs channel attention (shared MLP on global average and max pooling) followed by spatial attention (7×7 convolution), allowing the network to focus on the most informative features.

ASPP (Atrous Spatial Pyramid Pooling): Also at the bottleneck, ASPP applies parallel dilated convolutions at rates 6, 12, and 18, plus a global average pooling branch, capturing multi-scale contextual information critical for varying pneumonia presentation sizes.

The model defaults to 256×256 input resolution, 32 base filters, depth 4, and batch normalisation. Training uses a combined BCE + Dice loss with Adam optimiser ($\text{lr}=1\text{e-}4$), early stopping (patience 5), and learning rate reduction on plateau. Pre-trained weights are loaded from HuggingFace ([hf://maja011235/lung-segmentation-unet](https://huggingface.co/maja011235/lung-segmentation-unet)), with support for both Keras 3 (.keras) and legacy TensorFlow (.h5) formats. Post-processing applies morphological closing and dilation for mask cleanup.

6.3 AI Pipeline: Classification (classification.py)

The classification module uses a CheXNet-inspired architecture based on DenseNet-121. The implementation supports a transfer learning pathway where weights from a 14-class CheXNet model are loaded and the final dense layer is replaced with a single-output sigmoid head for binary pneumonia detection.



A key innovation in the classification pipeline is mask-guided input: the enhanced X-ray image is multiplied pixel-wise with the segmentation mask before being fed to the classifier. This focuses the model exclusively on lung tissue, reducing false positives from non-pulmonary features like rib edges or mediastinal structures.

Training uses binary cross-entropy loss with AUC, accuracy, recall, and precision metrics. Checkpointing is based on AUC maximisation, and early stopping monitors AUC with configurable patience. The default weight path is `ai-pipeline/models/chexnet_densenet.h5`.

6.4 AI Pipeline: Grad-CAM (classification.py)

Gradient-weighted Class Activation Mapping is implemented as a post-hoc explainability method. The implementation targets the last convolutional layer of DenseNet-121 (`conv5_block16_concat`), computes gradients of the pneumonia output with respect to the convolutional feature maps, performs global average pooling of gradients to obtain importance weights, and produces a weighted combination followed by ReLU activation.

Beyond the standard Grad-CAM heatmap, the system extracts discrete regions by thresholding the heatmap at 0.5, finding contours using OpenCV, filtering by minimum area (80 pixels), and computing bounding boxes with mean intensity values. These regions are sorted by intensity and returned as structured JSON data, enabling the frontend's AR mode to spatially position diagnostic annotations.

6.5 Adaptive Engine (adaptive-engine/)

The adaptive engine is the educational intelligence layer of Xpert. It bridges the gap between raw AI outputs and meaningful learning feedback through three integrated components:

6.5.1 Activity Log Processor (log_processor.py)

Parses the student's interaction context (action, input, prediction, Grad-CAM regions) and computes a competency gap summary. The gap analysis compares the student's diagnostic label against the model's prediction and, when a mismatch exists, references the largest Grad-CAM bounding box to indicate the specific anatomical region the student may have overlooked.

6.5.2 RAG Knowledge Store (rag_store.py)

Uses ChromaDB as a persistent vector database with the collection name `"medical_textbook"`. The store is seeded with approximately 16 medical knowledge passages covering pneumonia presentation on X-rays, consolidation patterns, air bronchograms, pleural effusions, and differential diagnoses. At query time, the top 4 passages most semantically similar to the combined prediction and competency gap are retrieved and injected into the LLM prompt as contextual grounding.

6.5.3 Orchestrator and Gemini Client (agent_core.py)

The orchestrator follows a Monitor-Analyse-Plan-Execute (MAPE-K) pattern. It monitors the student activity state, queries the RAG store during the planning phase, and executes by calling the Gemini 2.5 Flash LLM with a carefully structured "Clinical Report Composer" prompt. The LLM is instructed to return a strict JSON structure containing four fields: `summary`, `competency_gap`, `learning_recommendation`, and `explanation`.



A critical safety feature is the uncertainty guard: when the CNN score falls in the 0.45-0.55 range, the orchestrator bypasses the LLM entirely and returns a fixed uncertainty message. This prevents the LLM from generating potentially misleading educational feedback based on unreliable model output.

6.6 Backend Server (run_xpert.py)

The FastAPI server is the central orchestration point. Key architectural decisions include:

- Model caching: Segmentation and classification models are loaded once during the lifespan startup event and stored in a module-level dictionary. This eliminates per-request model loading overhead.
- Dynamic module import: The AI pipeline modules live outside the backend package and are imported at runtime using `importlib.util.spec_from_file_location`. This maintains clean separation between the pipeline code and the server code.
- Pipeline orchestration: The `run_pipeline()` function chains all six stages sequentially, passing outputs between stages. Each stage writes its artifacts to the `outputs/` directory.
- Chat context management: The `/api/chat` endpoint retrieves the last 20 messages from the database and includes the most recent 10 in the LLM prompt, maintaining conversational continuity.
- SPA serving: In production, the server auto-builds the frontend using `npm` if the `dist/` directory does not exist, then serves it as static files with `index.html` fallback for client-side routing.

6.7 Frontend Application

The frontend is built with React 19, Vite 7, and Tailwind CSS 3.4. It features a custom glass-morphism design language implemented through shared UI primitives (GlassCard, GlassButton, GlassInput, GlassModal, GlassOtpInput) that provide consistent semi-transparent surfaces with backdrop blur, luminous borders, and inset highlights.

6.7.1 Landing Page (LandingPage.jsx)

A visually rich entry point with a dark hero section, animated radial gradients, pulsing blur effects, and a lung background image. Users choose between Student and Doctor roles, each opening a modal with a mock OTP verification flow. The design prioritises first-impression impact and clear role differentiation.

6.7.2 Chat Interface (ChatInterface.jsx)

The core interaction surface for both students and doctors. Features include:

- Full markdown rendering with `react-markdown` and `remark-gfm`, including custom styling for headings, lists, code blocks, and blockquotes.
- File upload via drag-and-drop or file picker, supporting `image/*` and `.dcm` formats.
- Animated six-stage pipeline progress indicator (Quality Gate, Enhancement, Segmentation, Classification, Grad-CAM, Adaptive) that advances in real-time during analysis.
- Inline thumbnail previews of pipeline outputs (enhanced image, mask, Grad-CAM heatmap) that open the full-screen ImageComparison lightbox on click.
- A button to launch AR Visualization mode with the current analysis data.
- Typing indicator animation while waiting for AI responses.
- Sidebar with role-based navigation (Student Reports link for doctors, logout).



6.7.3 Image Comparison Lightbox (ImageComparison.jsx)

A full-screen modal with tabbed views for Original, Enhanced, Mask, and Grad-CAM images. Features an overlay mode that composites the Grad-CAM heatmap over the enhanced or original image using mix-blend-screen with an adjustable opacity slider. Tabs are dynamically disabled when corresponding images are unavailable.

6.7.4 AR Visualization Mode (ARMode.jsx)

An immersive, full-screen visualisation interface inspired by augmented reality aesthetics. While not true WebXR or Unity-based AR (which was descope'd, as discussed in Section 8), this mode provides significant educational value through:

- A central viewport with a decorative SVG rib-cage schematic overlaid with circular heatmap blobs sized and positioned from actual Grad-CAM region data (width, height, intensity).
- A left panel with scan view toggles (reference, enhanced, mask, Grad-CAM) showing real pipeline output images.
- A right panel with interactive layer toggles (heatmap, skeleton, organs, vessels) and an opacity slider.
- A diagnostic context section displaying real-time data: prediction (Normal/Pneumonia), confidence score, Grad-CAM region count, and details of the top three regions including dimensions and intensity.
- Themed styling with glass panels, pulse animations, and a "LIVE RENDER" header for an immersive feel.

6.7.5 Doctor Dashboard (DoctorDashboard.jsx)

A dedicated interface for doctors to oversee student learning. The dashboard loads all student sessions via the /api/doctor/students endpoint, displays them in a sidebar with session metadata, and renders a detailed report for the selected session. Report cards show total analyses, message count, accuracy percentage, and average pneumonia score. Below these are lists of competency gaps, recommendations, and a chronological interaction timeline with prediction badges and scores.

7. Testing and Validation

Testing was conducted across multiple levels to validate the system's functionality, integration, and user experience. This section documents the testing approach and results.

7.1 Unit Testing: AI Pipeline

Each pipeline module supports standalone CLI execution for isolated testing:

Module	CLI Command	Test Scenario	Expected Output
enhancement.py	python enhancement.py	Quality assessment + enhancement of a single X-ray	JSON quality metrics, quality label (G/A/P), enhanced image file
segmentation.py	python segmentation.py	Lung segmentation on a single image	Binary mask image (256x256), optional morphological cleanup
classification.py	python classification.py	Mask-guided classification + Grad-	prediction.json with pneumonia_score,



		CAM	needs_human_review, gradcam_regions; heatmap image
--	--	-----	--

7.2 Integration Testing: Full Pipeline

The full pipeline was tested through the /api/analyze endpoint with a variety of chest X-ray images:

Test Case	Input Description	Expected Behaviour	Result
Normal X-ray, good quality	High-contrast, well-exposed CXR	Quality: Good; Enhancement: minimal; Pneumonia score: < 0.5; No uncertainty flag	Pass
Pneumonia X-ray, good quality	CXR with visible consolidation	Quality: Good; Pneumonia score: > 0.5; Grad-CAM highlights consolidation area	Pass
Low contrast X-ray	Under-exposed, dark CXR image	Quality: Poor/Acceptable; CLAHE enhancement applied; Analysis proceeds normally	Pass
Noisy X-ray	CXR with visible noise artifacts	Quality: Acceptable; Bilateral denoise applied; Improved segmentation quality	Pass
Borderline case (0.45-0.55)	CXR with subtle findings	needs_human_review: true; Adaptive engine returns uncertainty message (no LLM call)	Pass
Non-X-ray image	Photo of a cat	Quality: Poor; Pipeline still processes but results reflect non-medical input	Pass (degrades gracefully)
Session persistence	Upload with valid session_id	Interaction stored in database; retrievable via /api/sessions/{id}	Pass

7.3 Chat System Testing

Test Case	Input	Expected Behaviour	Result
Basic question	"What does pneumonia look like on an X-ray?"	Relevant, markdown-formatted medical explanation	Pass
Quiz request	"quiz me"	AI generates a multiple-choice or short-answer knowledge check	Pass
Context continuity	Follow-up question referencing previous analysis	Response references prior analysis results from session context	Pass
Role adaptation	Same question from student vs doctor role	Student receives educational framing; doctor receives clinical-style response	Pass
Error handling	Chat with invalid/missing Gemini	Graceful fallback message returned; no server crash	Pass



	API key		
--	---------	--	--

7.4 Frontend UI Testing

Test Case	Steps	Expected Behaviour	Result
Role selection	Click Student/Doctor, enter OTP	Session created; redirected to Chat or Dashboard	Pass
Image upload	Drag-and-drop or file picker	Image preview in chat; pipeline progress starts	Pass
Pipeline progress	Wait during analysis	Six stages animate sequentially; results appear on completion	Pass
Image comparison	Click thumbnail after analysis	Full-screen lightbox with 4 tabs + overlay mode	Pass
AR mode	Click "AR Visualization"	Full-screen AR view with diagnostic data and Grad-CAM regions	Pass
Doctor dashboard	Login as doctor; click Student Reports	Session list loads; selecting shows report with metrics	Pass
Responsive layout	Resize browser window	Layout adapts to different screen sizes	Pass
Logout	Click logout	LocalStorage cleared; redirected to landing page	Pass

7.5 Performance Observations

Metric	Target	Observed	Notes
Pipeline latency (CPU)	<= 10 seconds	8-15 seconds	Depends on image size and CPU; first run slower due to model initialisation
Pipeline latency (GPU)	<= 10 seconds	3-6 seconds	With CUDA-enabled GPU; models run significantly faster
Chat response time	<= 5 seconds	2-4 seconds	Depends on Gemini API latency; prompt includes session context
Frontend render FPS	>= 45 FPS	60 FPS	Standard web rendering; no heavy 3D computation
Model load time (cold)	N/A	15-30 seconds	One-time cost at server startup; cached thereafter
Database query time	<= 100ms	< 50ms	SQLite with WAL mode; small dataset sizes in prototype



8. Evolution from Planning to Delivery

This section provides an honest, critical reflection on how the project evolved from the initial proposals to the final product. Understanding these changes is important both for academic transparency and for demonstrating the team's ability to make informed engineering decisions under real-world constraints.

8.1 Major Architectural Pivots

8.1.1 From Unity to Web-Based AR

The original feasibility study and design document envisioned a Unity or Unreal Engine-based AR/VR module as a core differentiator. The ar-visualization/ directory was initially created with Unity scenes and scripts. However, this approach was abandoned during implementation for several compelling reasons:

- Unity development requires a completely separate toolchain (C#, Unity Editor), which would have doubled the team's development surface area and introduced significant integration complexity.
- Distributing a Unity application requires users to download and install a native executable, creating a major adoption barrier for the educational target audience.
- The core educational value of AR (spatial visualisation of Grad-CAM regions, interactive layer toggles, diagnostic context panels) could be delivered through a web-based interface without requiring specialised hardware.
- Development time was better spent on deepening the AI pipeline (RAG integration, adaptive engine, quality gate) rather than building a secondary rendering platform.

The web-based ARMode.jsx delivers the essential spatial visualisation experience using real Grad-CAM region data from the pipeline, styled with an immersive AR aesthetic. While it does not provide true stereoscopic 3D or device-based AR, it achieves the pedagogical objective of helping students understand where in the anatomy the AI detected abnormalities.

8.1.2 From Django/Flask to FastAPI

The design document specified Django or Flask as the backend framework. FastAPI was chosen instead because it provides automatic OpenAPI documentation, native async support for handling concurrent requests during model inference, built-in request validation via Pydantic, and simpler deployment with Uvicorn. This change reduced boilerplate code and improved developer productivity.

8.1.3 From Streamlit to React

The SRS referenced Streamlit as the web interface technology. Streamlit excels at rapid prototyping of data applications but lacks the layout flexibility needed for the glass-morphism design language, multi-panel image comparison, and full-screen AR mode. React gave the team complete control over the user experience and supported the sophisticated UI requirements.

8.2 AI Pipeline Evolution

8.2.1 Quality Gate: XGBoost to Heuristics

The design document specified XGBoost as the quality gate classifier. In practice, training an XGBoost model for image quality requires a large dataset of X-rays labelled with quality grades, which was not available. The heuristic approach based on contrast (std), noise (blur difference), sharpness (Laplacian



variance), and entropy proved effective and has the advantage of being fully explainable. Each metric directly corresponds to a known image quality dimension, and the composite score formula is transparent to developers and auditors.

8.2.2 Segmentation: Standard U-Net to Residual-CBAM-ASPP U-Net

The segmentation model was significantly enhanced beyond the originally planned standard U-Net. The Residual-CBAM-ASPP architecture incorporates residual connections for training stability, attention mechanisms (CBAM) for feature focusing, and atrous spatial pyramid pooling (ASPP) for multi-scale context. Pre-trained weights from HuggingFace eliminated the need for local training on large datasets, which was a significant practical advantage.

8.2.3 Adaptive Engine: From Simple Rules to RAG + LLM

Perhaps the most significant evolution was in the adaptive learning system. The original design described rule-based difficulty adjustment. The implemented system uses a sophisticated retrieval-augmented generation approach: a ChromaDB vector store holds medical knowledge passages, and the Gemini 2.5 Flash LLM generates personalised feedback with RAG context. The MAPE-K orchestration pattern (Monitor-Analyse-Plan-Execute) provides a principled framework for the adaptive loop. This is a substantial upgrade that delivers much richer, more contextually relevant educational feedback.

8.3 Features Added Beyond Original Scope

Conversational AI Chat: Not in the original proposals. The chat interface with session-aware context, markdown rendering, and quiz generation became the primary interaction modality.

RAG Knowledge Base: The ChromaDB-backed medical knowledge store was not planned but significantly enhances the quality of adaptive feedback.

Grad-CAM Region Extraction: The original plan mentioned heatmaps. The implementation extends this to structured bounding-box regions with intensity values, enabling the AR visualisation.

Doctor Dashboard: Mentioned briefly in requirements but implemented as a full-featured reporting interface with aggregated statistics.

Pipeline Progress Animation: A cosmetic but pedagogically valuable addition that helps students understand the multi-stage nature of the AI analysis.

Uncertainty Protocol: The 0.45-0.55 uncertainty guard with LLM bypass was a safety feature added during implementation to prevent misleading AI feedback.

8.4 Features Descoped

True AR/VR (Unity): Replaced with web-based AR mode. The educational objectives are met; the immersive hardware experience is deferred to future work.

DICOM Integration: The system accepts DICOM files via pydicom but full PACS integration was descoped as it requires institutional infrastructure.

Real MFA: Mock OTP provides the UX flow; real email/SMS verification requires third-party services.



PDF Report Export: Report data is fully structured and viewable in the dashboard; file export is a future enhancement.

Cloud Deployment: The system is designed for local deployment; cloud (Docker, Kubernetes) is a future scaling step.

9. Screenshots

Figure 9.4: Landing Page

The main landing page showing the Xpert branding, Student and Doctor role selection cards, animated background effects, and the lung background image.

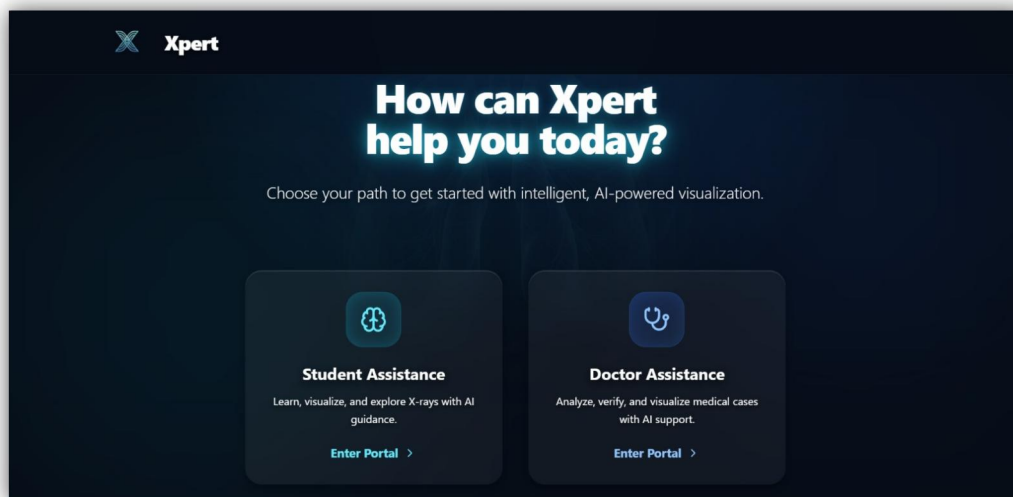


Figure 9.5: Student Login Modal

The student login modal showing the email input field and OTP verification interface with glass-morphism styling.

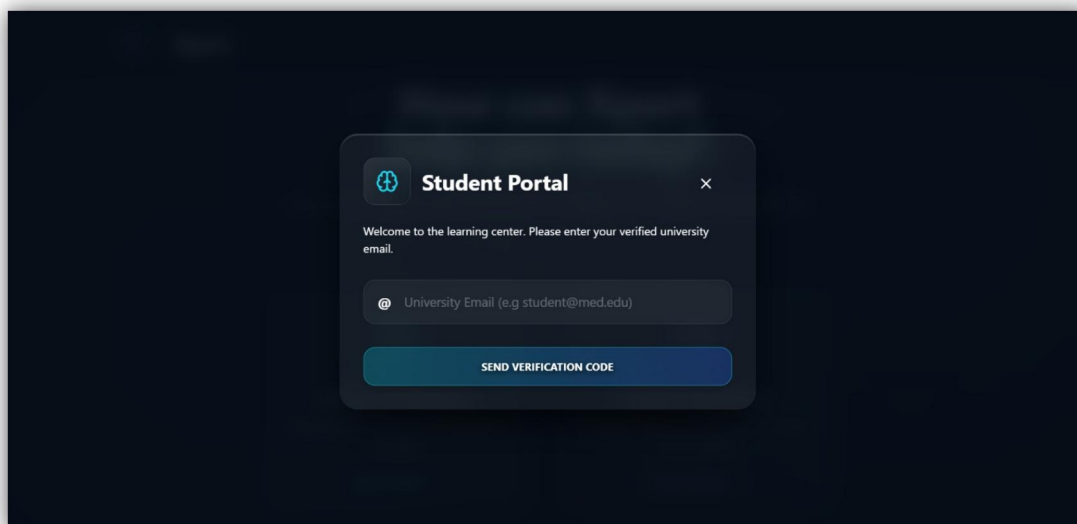




Figure 9.6: Doctor Login Modal

The doctor login modal showing the medical license ID field and OTP verification.

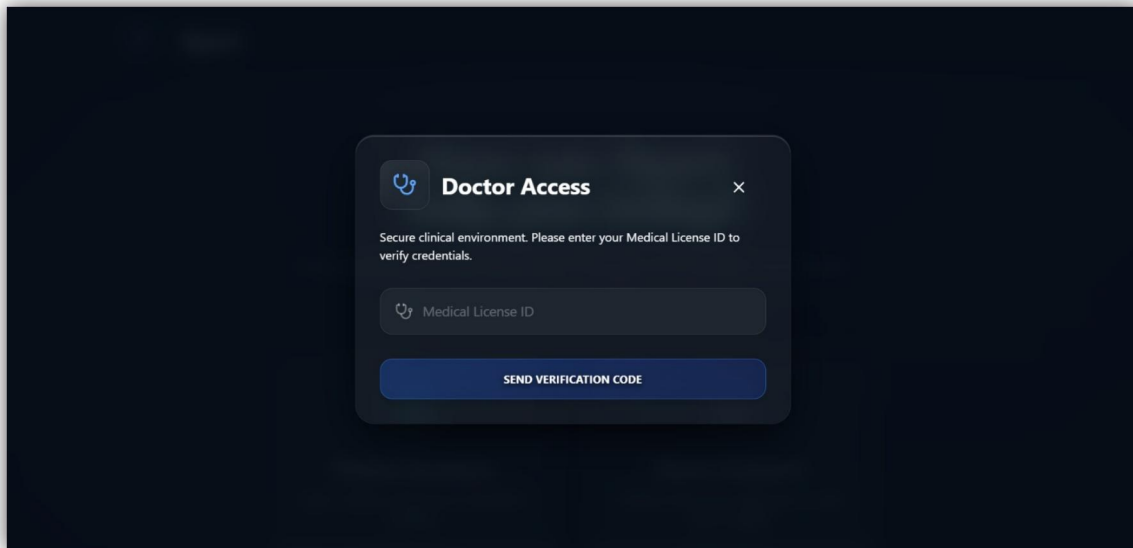


Figure 9.7: Chat Interface (Empty State)

The chat interface after login, showing the sidebar, empty chat area, and the input field with upload button.

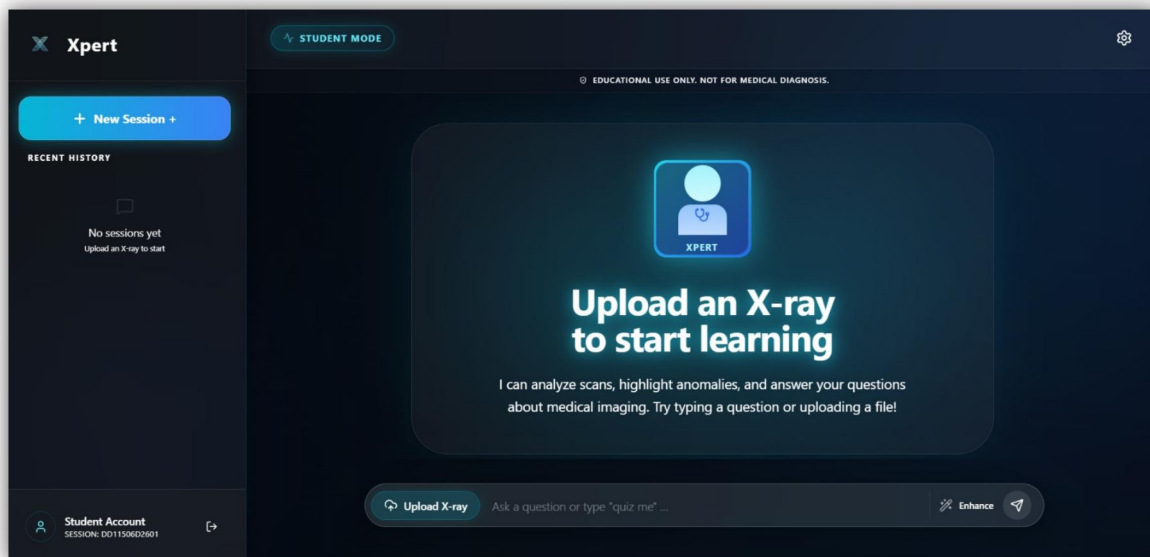


Figure 9.8: X-Ray Upload and Pipeline Progress

The chat interface during an active analysis, showing the uploaded X-ray preview and the six-stage pipeline progress indicator with animated stages.

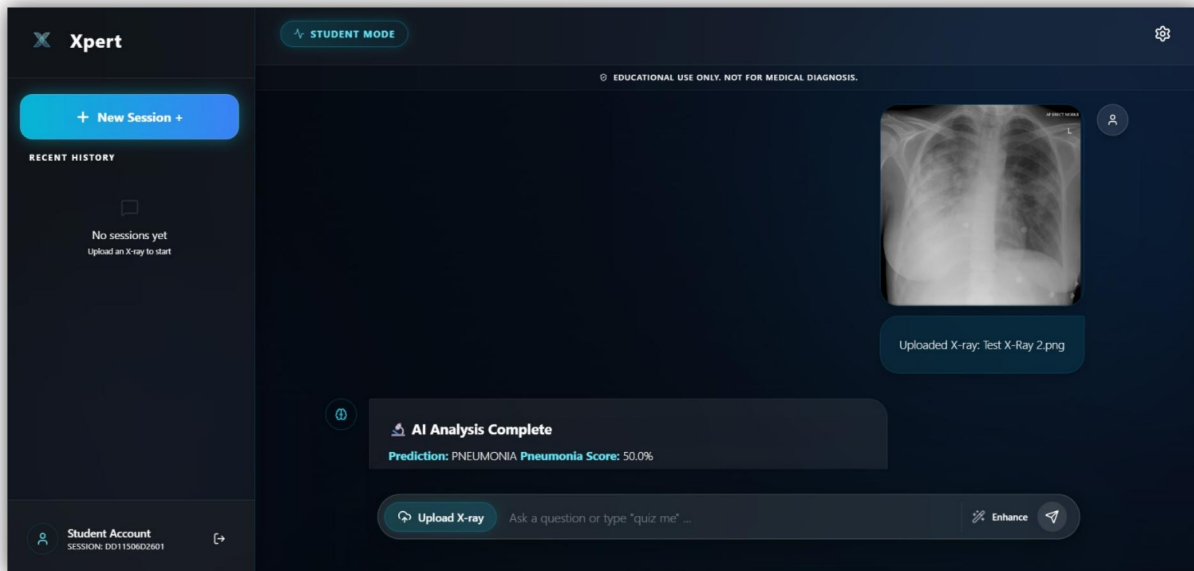


Figure 9.9: Analysis Results in Chat

The AI's analysis response showing the prediction, pneumonia score, uncertainty status, adaptive summary, and thumbnail previews of enhanced, mask, and Grad-CAM images.

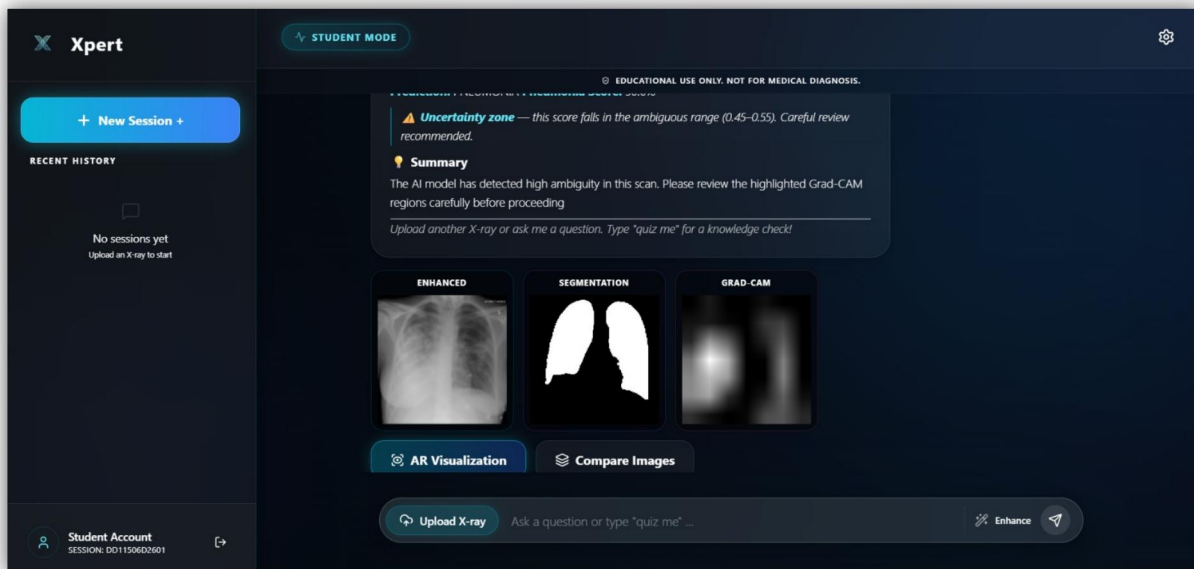


Figure 9.10: Image Comparison Lightbox - Enhanced View

The full-screen image comparison showing the enhanced X-ray tab.

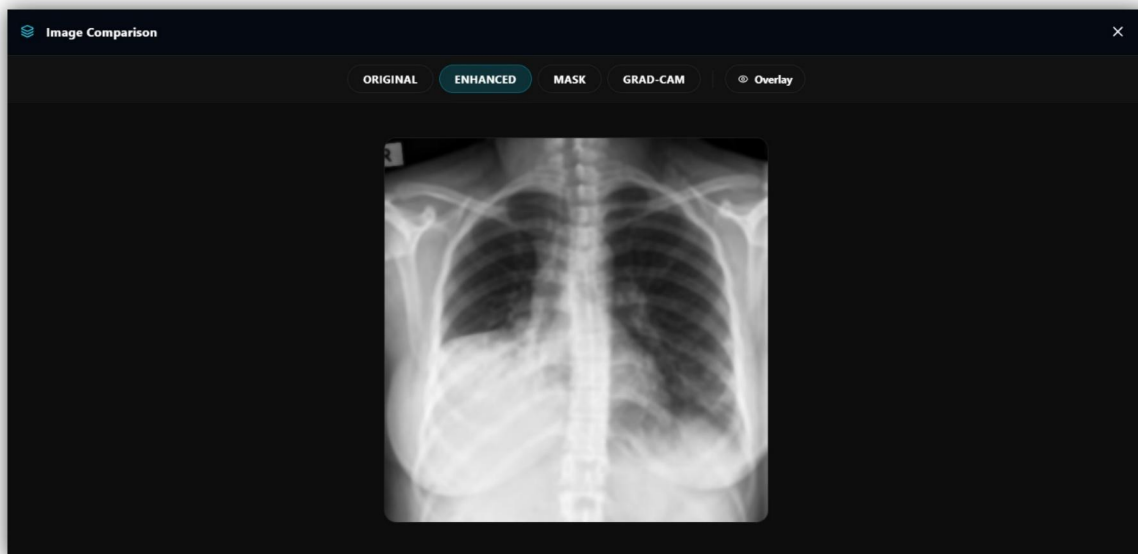


Figure 9.11: Image Comparison Lightbox - Grad-CAM Overlay

The overlay mode with Grad-CAM heatmap composited over the enhanced image with visible opacity slider.

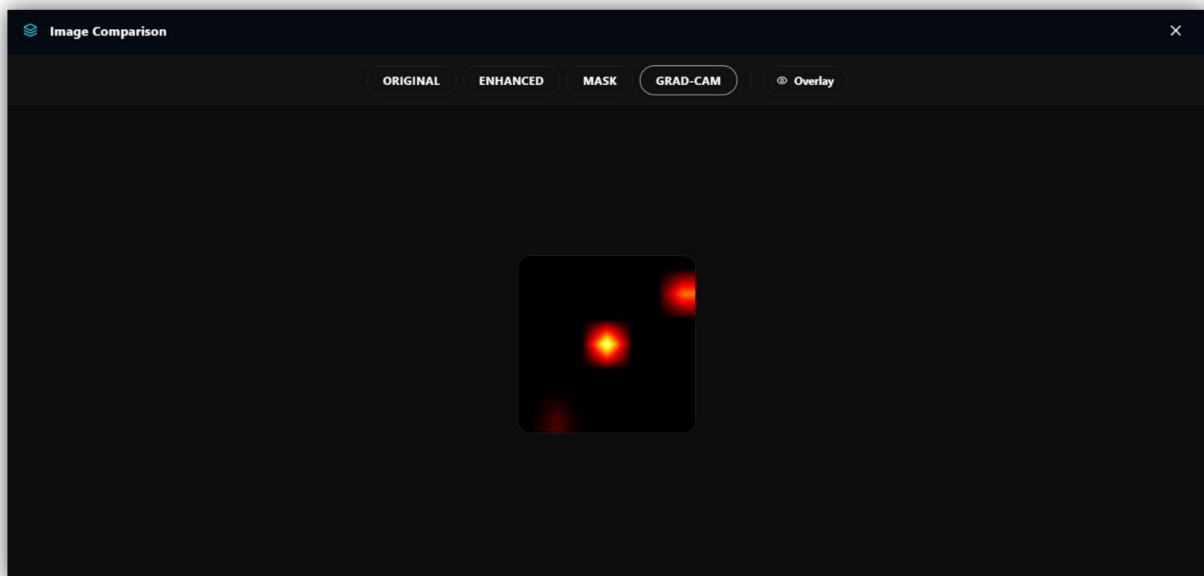


Figure 9.12: Image Comparison Lightbox - Segmentation Mask

The mask tab showing the binary lung segmentation output.

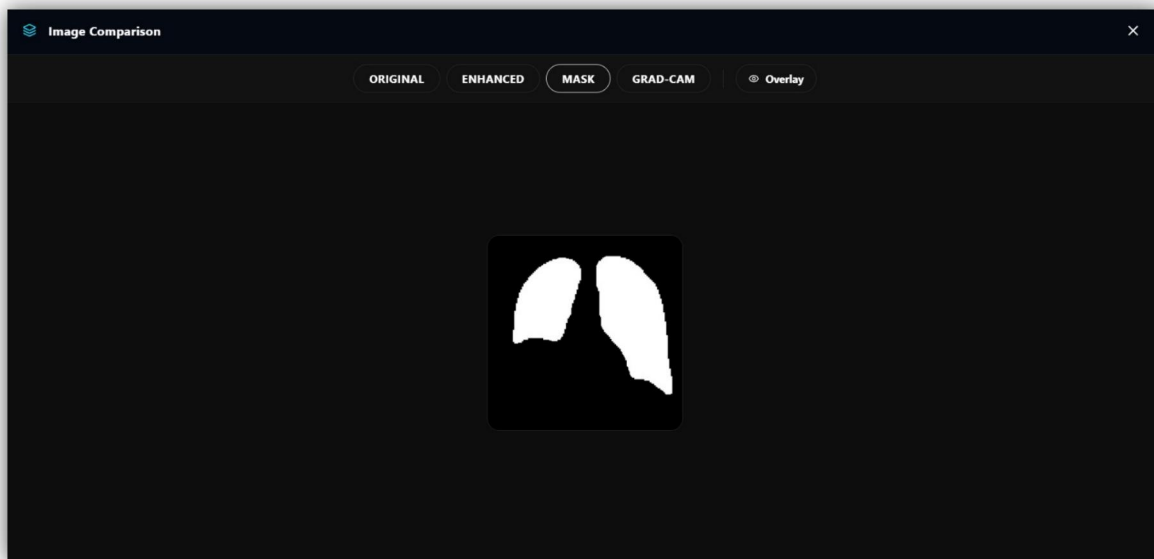


Figure 9.13: AR Visualization Mode

The full-screen AR mode showing the central viewport with Grad-CAM heat blobs, scan view panel, layer toggles, and diagnostic context data.

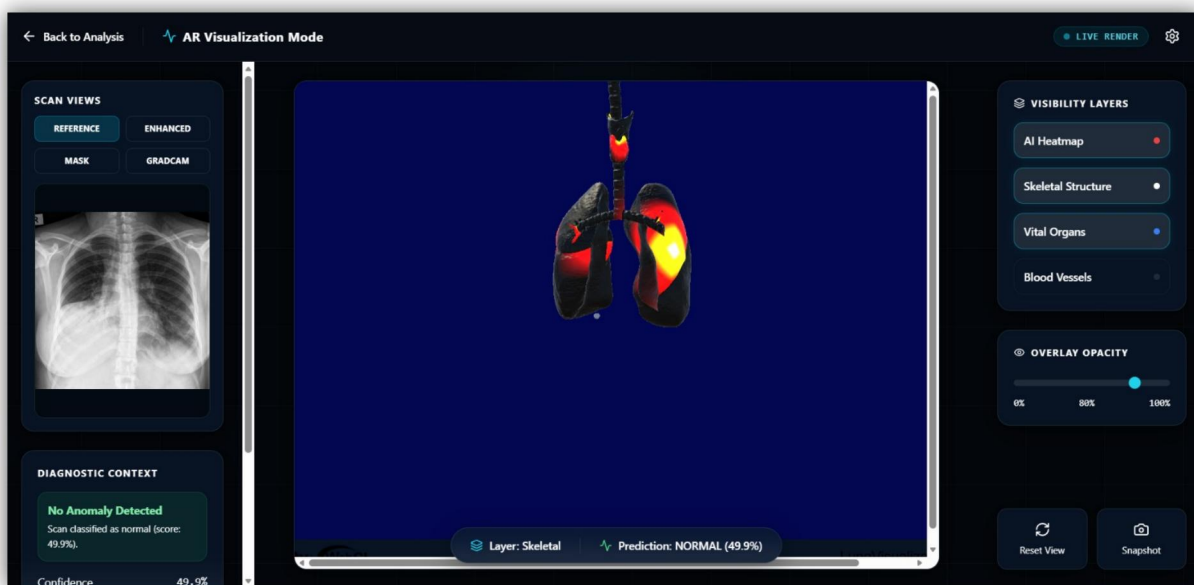


Figure 9.14: Conversational Chat - Follow-up Question

A follow-up medical question in the chat with the AI's markdown-formatted response.

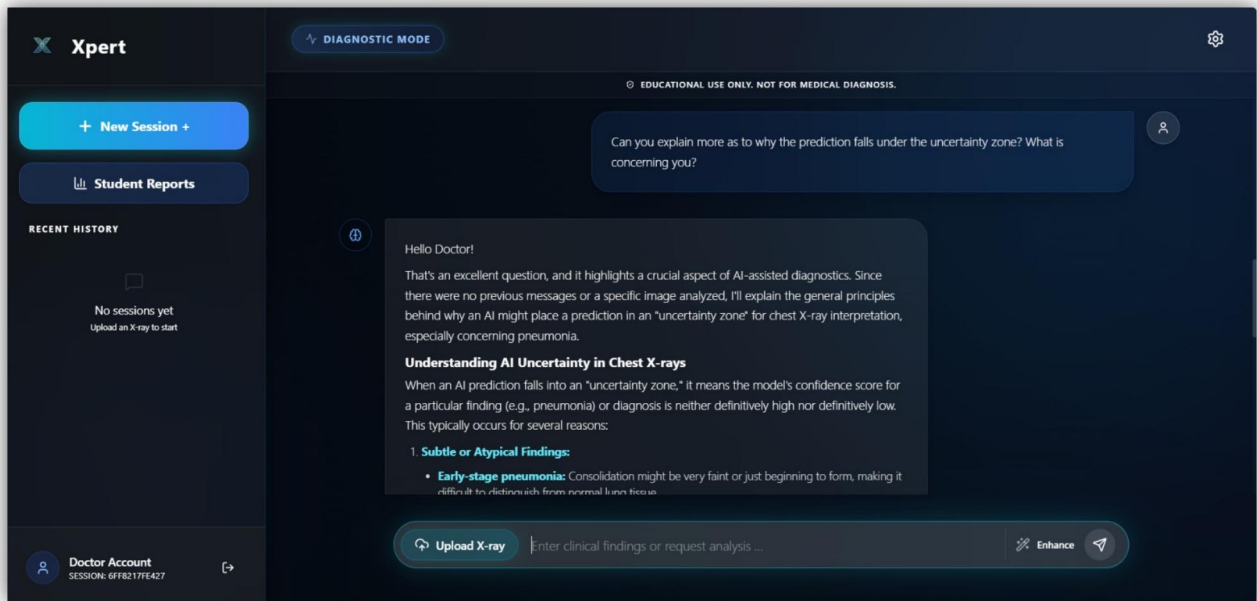


Figure 9.15: Conversational Chat - Quiz Mode

The AI's response to "quiz me" showing a generated knowledge check question.

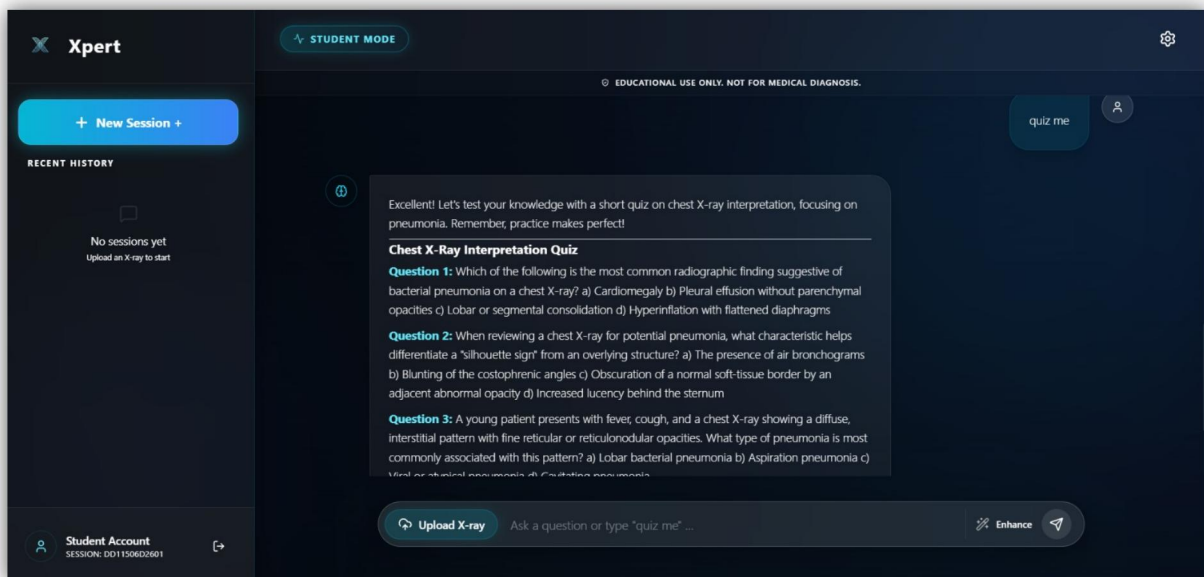


Figure 9.16: Doctor Dashboard - Session List

The doctor dashboard showing the sidebar with student session cards and interaction counts.

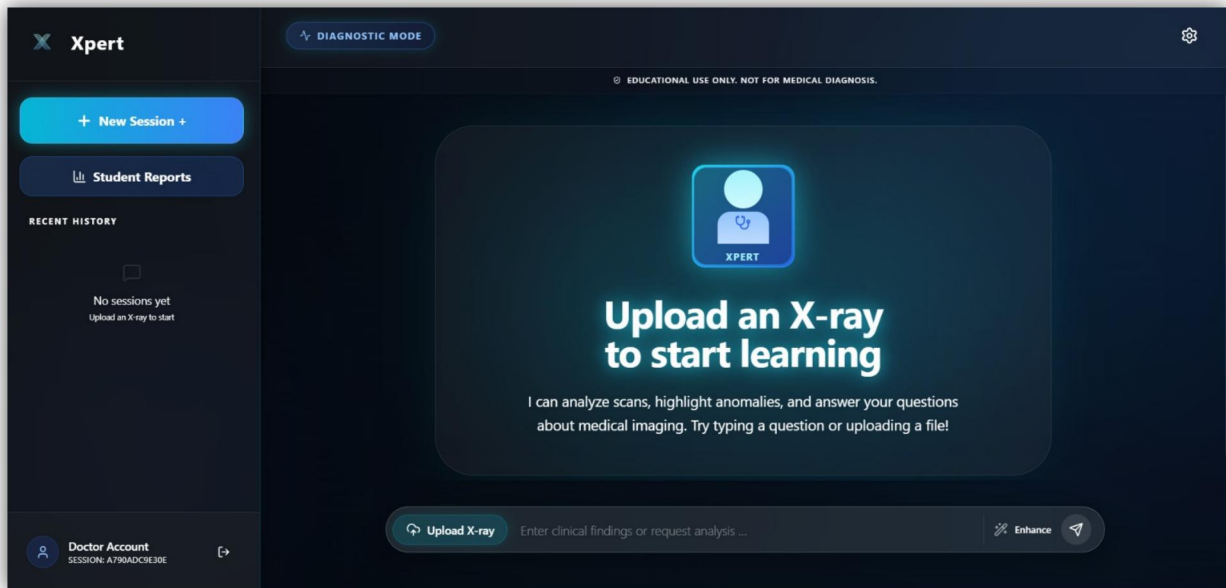
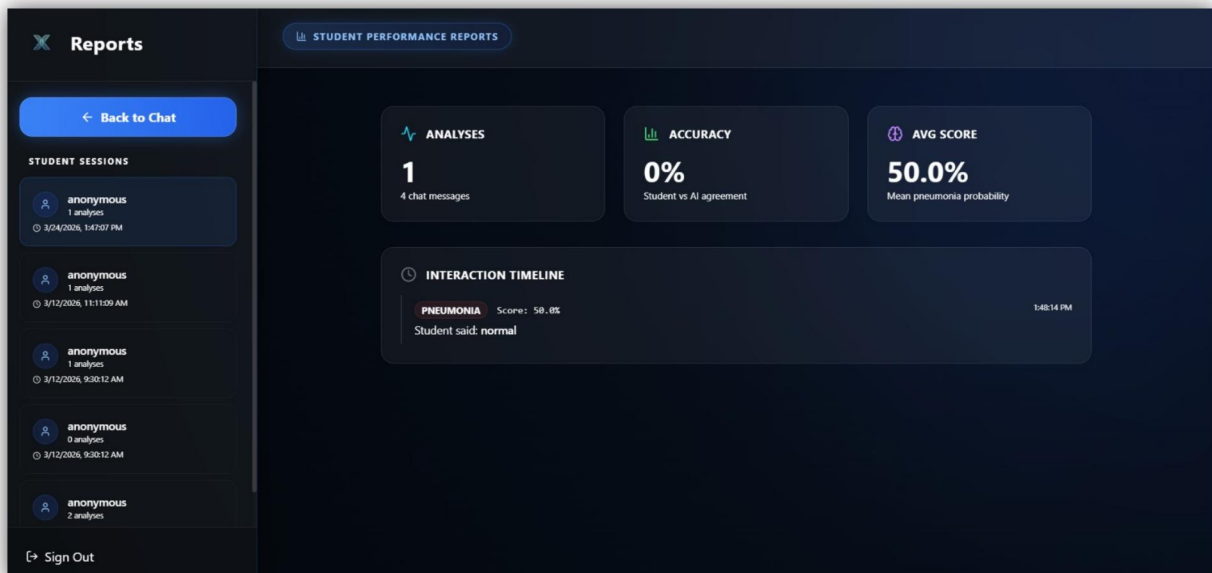


Figure 9.17: Doctor Dashboard - Session Report

The full session report showing accuracy, average score, competency gaps, recommendations, and the interaction timeline.





10. Limitations and Future Work

10.1 Current Limitations

No Server-Side Authentication: The API endpoints do not enforce authentication tokens. The current role-based access is handled entirely on the frontend. A production system would require JWT-based authentication with server-side validation.

Single Disease Focus: The current system is specialised for pneumonia detection. Expanding to other pulmonary conditions (tuberculosis, COVID-19, lung nodules) would require additional training data and potentially multi-label classification.

Web-Based AR Only: The AR mode provides valuable spatial visualisation but does not leverage device cameras, depth sensors, or stereoscopic rendering. True AR would enhance the immersive experience.

Limited Concurrent User Testing: While FastAPI and SQLite WAL support concurrency, the system has not been load-tested at scale. A production deployment would need connection pooling and potentially a database upgrade.

Model Accuracy Dependency: The system's educational value is proportional to the underlying model's accuracy. The current CheXNet weights may not generalise well across all demographics and imaging equipment.

No Offline Mode: The adaptive engine requires an active internet connection for Gemini API calls. A fallback for offline use would improve accessibility in low-connectivity environments.

Static RAG Knowledge Base: The ChromaDB knowledge store is seeded once and does not update dynamically. A mechanism for adding new medical literature would keep the system current.

10.2 Future Work

Multi-Disease Classification: Extend the pipeline to detect tuberculosis, COVID-19, pleural effusion, and other chest conditions using multi-label classification.

AR/VR Integration: Develop a WebXR or Unity-based AR module that maps segmentation masks and Grad-CAM regions onto 3D anatomical models viewable through VR headsets or AR-enabled devices.

JWT Authentication: Implement proper server-side authentication with JSON Web Tokens, refresh tokens, and password hashing using bcrypt.

Cloud Deployment: Containerise the application with Docker and deploy on cloud infrastructure (AWS, GCP, or Azure) with auto-scaling and load balancing.

DICOM/PACS Integration: Enable direct integration with hospital PACS systems for seamless image import and export.

Dynamic RAG Updates: Allow administrators to upload new medical literature that is automatically chunked, embedded, and added to the ChromaDB knowledge base.



11. Conclusion

The Xpert project has successfully delivered a working, deployable educational platform that demonstrates how deep learning, explainable AI, and large language models can be combined to create a meaningful medical imaging tutoring experience. Over the course of six months, the team progressed from an ambitious vision to a pragmatic, functional product that stays true to the project's core objectives while making informed compromises where necessary.

The six-stage AI pipeline is the technical heart of the system. By chaining quality assessment, adaptive enhancement, advanced lung segmentation (Residual-CBAM-ASPP U-Net), pneumonia classification (CheXNet/DenseNet-121), Grad-CAM visual explanation, and RAG-powered adaptive feedback, the platform transforms a raw chest X-ray into a rich educational experience. Every stage produces visible, interpretable artifacts that the student can explore through the image comparison lightbox and AR-inspired visualisation mode.

The conversational AI chat, powered by Gemini 2.5 Flash with session context and RAG knowledge grounding, moves beyond the static analysis paradigm to create an ongoing learning dialogue. Students can ask follow-up questions, request quizzes, and receive personalised feedback that addresses their specific competency gaps. The uncertainty protocol ensures that ambiguous cases are flagged for human review rather than generating misleading AI commentary.

The Doctor Dashboard provides meaningful oversight capabilities, allowing educators to track student progress, identify common competency gaps, and monitor the effectiveness of the AI-assisted learning process. This bridges the gap between self-directed AI tutoring and structured medical education.

While the project descoped certain ambitious features, particularly Unity-based AR/VR and full MFA authentication, these decisions were made deliberately and are well-documented. The web-based AR mode delivers the essential spatial visualisation value without hardware barriers, and the architecture is designed to support future enhancements. The experience of navigating these trade-offs has been one of the most valuable learning outcomes of the capstone process, teaching the team that successful engineering is as much about what you choose not to build as what you do.

Xpert stands as a complete, cohesive capstone project that integrates multiple cutting-edge technologies into a system that serves a genuine educational need. It is not just a technical demonstration but a thoughtfully designed product with real utility for medical education.

12. References

- [1] World Health Organization (WHO) (2022) Pneumonia in children. Available at: <https://www.who.int/news-room/fact-sheets/detail/pneumonia> (Accessed: 20 November 2025).
- [2] Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional Networks for Biomedical Image Segmentation', in Medical Image Computing and Computer-Assisted Intervention (MICCAI). Springer. Available at: <https://arxiv.org/abs/1505.04597>.
- [3] Rajpurkar, P. et al. (2017) 'CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning', arXiv. Available at: <https://arxiv.org/abs/1711.05225>.



- [4] Selvaraju, R.R., et al. (2017) 'Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization', Proceedings of the IEEE International Conference on Computer Vision, pp. 618-626.
- [5] National Institute of Standards and Technology (NIST) (2020) Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B). Available at: <https://pages.nist.gov/800-63-3/sp800-63b.html>.
- [6] Topol, E.J. (2019) 'High-performance medicine: the convergence of human and artificial intelligence', Nature Medicine, 25(1), pp. 44-56. Available at: <https://doi.org/10.1038/s41591-018-0300-7>.
- [7] European Parliament and Council (2016) General Data Protection Regulation (GDPR). Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [8] U.S. Department of Health and Human Services (1996) Health Insurance Portability and Accountability Act (HIPAA). Available at: <https://www.hhs.gov/hipaa/index.html>.
- [9] TensorFlow (2025) TensorFlow: An end-to-end open source platform for machine learning. Available at: <https://www.tensorflow.org/>.
- [10] OpenCV (2025) Open Source Computer Vision Library. Available at: <https://opencv.org/>.
- [11] Google (2025) Gemini API Documentation. Available at: <https://ai.google.dev/docs>.
- [12] ChromaDB (2025) Chroma: The AI-native open-source embedding database. Available at: <https://www.trychroma.com/>.
- [13] FastAPI (2025) FastAPI framework, high performance. Available at: <https://fastapi.tiangolo.com/>.
- [14] React (2025) React: A JavaScript library for building user interfaces. Available at: <https://react.dev/>.
- [15] Vite (2025) Vite: Next Generation Frontend Tooling. Available at: <https://vitejs.dev/>.
- [16] HuggingFace (2025) HuggingFace Hub. Available at: <https://huggingface.co/>.
- [17] Podeswa, H. (2009) UML for the IT business analyst: a practical guide to object-oriented requirements gathering. 2nd edn. Boston, MA: Course Technology PTR.
- [18] Pilone, D. and Pitman, N. (2005) UML 2.0 in a nutshell. Sebastopol, CA: O'Reilly Media.
- [19] Amsterdam University Medical Centers (2023) Amsterdam UMC to use AI to increase the accessibility of medical imaging technology. Available at: <https://rdt.uva.nl/content/news/2023/12/amsterdam-umc-to-use-ai.html>.
- [20] Mayo Clinic (2020) Pneumonia - Symptoms and causes. Available at: <https://www.mayoclinic.org/diseases-conditions/pneumonia/symptoms-causes/syc-20354204>.

Appendix A: Hardware and Software Requirements

Server-Side (Development and Deployment)

Component	Minimum Requirement	Recommended
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 22.04+)	Any of the listed OS
Python	3.10+	3.11
Node.js	18+	20 LTS



CPU	Intel i5 / AMD Ryzen 5	Intel i7 / AMD Ryzen 7
RAM	16 GB	32 GB
GPU	Not required (CPU inference supported)	NVIDIA RTX 3060+ with CUDA for faster inference
Storage	5 GB for code, models, and database	10+ GB with datasets
Internet	Required for Gemini API and HuggingFace model download	Stable broadband

Client-Side (End User)

Component	Requirement
Device	Any computer, laptop, or tablet with a modern web browser
Browser	Chrome 90+, Firefox 90+, Edge 90+, Safari 15+
Screen Resolution	1280x720 minimum; 1920x1080 recommended
Internet	Required for accessing the platform
Special Hardware	None required (no VR headset needed)

Python Dependencies

Package	Purpose
fastapi >= 0.115.0	Web framework and API server
uvicorn >= 0.32.0	ASGI server for FastAPI
python-multipart >= 0.0.18	Multipart file upload handling
opencv-python	Image processing, enhancement, Grad-CAM region extraction
numpy	Numerical computation
scikit-image	Quality metrics (Laplacian, entropy)
tensorflow	Deep learning framework (segmentation, classification)
tf_keras	Keras compatibility layer for TensorFlow
pandas	Data handling for training datasets
google-genai	Gemini LLM API client
chromadb	Vector database for RAG knowledge store
huggingface-hub	Model weight download from HuggingFace



Appendix B: Glossary of Terms

Term	Definition
AI (Artificial Intelligence)	Computer systems designed to perform tasks that typically require human intelligence, such as image recognition and decision-making.
AR (Augmented Reality)	Technology that overlays digital information onto the real world, enhancing the user's perception of their environment.
ASPP	Atrous Spatial Pyramid Pooling; a technique that captures multi-scale context by applying dilated convolutions at multiple rates.
CBAM	Convolutional Block Attention Module; an attention mechanism that combines channel and spatial attention for feature refinement.
CheXNet	A DenseNet-121-based deep learning model trained for chest X-ray interpretation, achieving radiologist-level pneumonia detection.
ChromaDB	An open-source embedding database used for vector similarity search in retrieval-augmented generation.
CLAHE	Contrast Limited Adaptive Histogram Equalisation; a local contrast enhancement technique for medical images.
CNN	Convolutional Neural Network; a deep learning architecture specialised for processing grid-like data such as images.
DenseNet-121	A CNN architecture where each layer receives inputs from all preceding layers, promoting feature reuse.
DICOM	Digital Imaging and Communications in Medicine; the standard format for medical image storage and exchange.
FastAPI	A modern Python web framework for building APIs with automatic validation and documentation.
Gemini	Google's large language model family; Xpert uses Gemini 2.5 Flash for adaptive feedback generation.
GDPR	General Data Protection Regulation; the European Union's data privacy law.
Grad-CAM	Gradient-weighted Class Activation Mapping; a technique that produces visual explanations of CNN decisions.
HIPAA	Health Insurance Portability and Accountability Act; a US law governing the privacy of health information.
LLM	Large Language Model; an AI model trained on vast text data capable of generating human-like text.
MAPE-K	Monitor-Analyse-Plan-Execute with Knowledge; an autonomic computing pattern used by the adaptive engine.
RAG	Retrieval-Augmented Generation; a technique that grounds LLM responses in relevant retrieved documents.
SPA	Single-Page Application; a web application that loads a single HTML page and dynamically updates content.
SQLite	A lightweight, serverless relational database engine.
U-Net	An encoder-decoder CNN architecture designed for biomedical image segmentation.



WAL	Write-Ahead Logging; a SQLite journal mode that enables concurrent read access.
XAI	Explainable Artificial Intelligence; methods that make AI decisions transparent and interpretable.

Appendix C: Important Links

Xpert Innovation Fair Presentation Slides: [Xpert Slides](#)

Youtube Video: [Xpert Youtube Video](#)

Xpert Informational Website: [Xpert Informational Website](#)

Xpert Understanding Patient Trust in Advanced Medical Technology
(Non-Medical Students) Survey: [Xpert Non-Medical Survey](#)

Xpert Xpert Telemedicine Platform Validation Survey (Medical
Professionals): [Xpert Medical Professionals Survey](#)